HTTP Signature-Key Header

draft-hardt-httpbis-signature-key

Dick Hardt (Hellō) & Thibault Meunier (Cloudflare)

IETF 125 Shenzhen — March 2026

Agenda

1. Why HTTP Message Signatures?
2. The Missing Piece: Key Distribution
3. Signature-Key Header Field
4. Schemes: `hwk`, `jwks_uri`, `jwt`, `x509`
5. Spanning Schemes
6. Security & Privacy Considerations
7. IANA Registrations
8. **New**: `jkt-jwt` — Self-Issued Key Delegation

Why HTTP Message Signatures?

Limitations of other mechanisms

- **Bearer tokens** — bearer credential; anyone with the token can use it
- **DPoP** — proof of possession, but narrow scope (OAuth access tokens only)
- **mTLS** — typically terminated at load balancer; client identity lost before reaching the application; operationally burdensome (cert provisioning, rotation)

HTTP Message Signatures (RFC 9421)

- Proof of possession (and identity) at the application layer
- Signs message content (selective coverage)
- Works through proxies and CDNs

The Missing Piece: Key Distribution

- RFC 9421 defines HTTP Message Signatures — creation and verification
- But leaves key distribution to application protocols
- Verifiers need the signer's public key to verify a signature
- No standard way to distribute keys inline with HTTP messages

Signature-Key Header Field

- New HTTP header: `Signature-Key`
- Structured Fields Dictionary (RFC 8941), keyed by signature label
- Correlates with `Signature-Input` and `Signature` by label
- Carries the public key or a reference to it

```
Signature-Input: sig=("@method" "@authority" "@path" "signature-key"); created=1732210000
Signature:       sig=:MEQCIA5...
Signature-Key:   sig=hwk;kty="OKP";crv="Ed25519";x="JrQLj..."
```

How It Fits Together

- Labels (`sig`) correlate across all three headers
- `Signature-Key` value is: `<label>=<scheme>;<parameters>...`
- The **scheme token** (`hwk` , `jwks_uri` , `jwt` , `x509`) where the verifier obtains the key
- Verifier selects the `Signature-Key` member matching the label being verified

```
Signature-Key:   sig=hwk;kty="OKP";crv="Ed25519";x="JrQLj..."
                 ^^^ ^^^
                label scheme
```

Four Schemes

Scheme	Model	Identity	Key Discovery
hwk	Pseudonymous	Key thumbprint	None (inline)
jwks_uri	Identified	HTTPS URL	Well-known metadata
jwt	Delegated	JWT issuer	Issuer's JWKS
x509	PKI	Certificate subject	Certificate chain

hwk — Header Web Key

- Self-contained public key inline in the header
- Parameters map directly to JWK (RFC 7517): `kty`, `crv`, `x`, `y`, `n`, `e`
- Supports OKP, EC, and RSA key types
- No identity disclosure — pseudonymous verification
- Use cases: privacy-preserving agents, rate limiting per-key, Trust On First Use (TOFU)

```
Signature-Key: sig=hwk;kty="OKP";crv="Ed25519";x="JrQLj5P..."
```

jwks_uri — JWKS URI Discovery

- Identifies the signer via HTTPS URL
- Key discovery via well-known metadata document
- Parameters: `id`, `well-known`, `kid`
- Discovery: fetch metadata → extract `jwks_uri` → fetch JWKS → find `kid`
- Use cases: identified services, crawlers, monitoring

```
Signature-Key: sig=jwks_uri;id="https://service.example";well-known="oauth-authorization-server";kid="key-1"
```

jwt — JWT Confirmation Key

- Embeds public key in a signed JWT via `cnf.jwk` claim (RFC 7800)
- Enables delegation: authority signs JWT, instance signs HTTP messages
- JWT obtained out of band; verified via issuer's JWKS
- Use cases: horizontal scaling, ephemeral instance keys, delegation

```
Signature-Key: sig=jwt;jwt="eyJhbGciOiJFUzI1NiI..."
```

x509 — X.509 Certificates

- Certificate-based verification with PKI trust chains
- Parameters: `x5u` (cert URL), `x5t` (SHA-256 thumbprint)
- Full certificate chain validation, revocation checking
- Caching by thumbprint
- Use cases: enterprise PKI, regulated industries

```
Signature-Key: sig=x509;x5u="https://example/.well-known/cert.pem";x5t=:bWco...nM=:
```

Spanning Schemes

A workload calls an API:

1. **First contact** (`hwk`): workload sends a signed request with an inline key — no prior registration needed. API rate-limits and builds reputation per key thumbprint (TOFU)
2. **Established identity** (`jwks_uri`): workload registers an identity URL. API discovers the key via well-known metadata — verified, identified caller
3. **Delegated at scale** (`jwt`): platform issues JWTs to workload instances, each with an ephemeral `cnf` key. Instances sign requests independently — horizontal scale with centralized trust

Security Considerations

- **Signature-Key integrity**: cover `signature-key` in Signature-Input
  - Without it, attackers can substitute schemes or identities
    - *creates challenges for multiple signatures*
- **Key validation**: required for all schemes before use
- **Caching**: MAY cache keys with appropriate TTLs; MUST limit cache size
- **Algorithm selection**: verifiers MUST validate algorithm against policy

Privacy Considerations

- **hwk**: pseudonymous — no identity linkage (unless keys reused)
- **jwks_uri / x509 / jwt**: reveal signer identity
- **Key discovery tracking**: fetching from signer URLs creates tracking vectors
  - Mitigate with caching and shared infrastructure
- **JWT contents**: may contain claims beyond `cnf` — minimize retention

IANA Registrations

- **HTTP Field Name**: `Signature-Key` header
- **New Registry**: "HTTP Signature-Key Scheme"
  - Registration policy: Specification Required
  - Initial entries: `hwk`, `jwks_uri`, `x509`, `jwt`

New Scheme: jkt-jwt — Self-Issued Key Delegation

- **New scheme** (post-cutoff): self-issued key delegation via JWT

The problem

- Many devices have hardware-backed secure enclaves (Apple Secure Enclave, Android StrongBox, TPM)
- Enclaves can generate key pairs where the private key never leaves the hardware
- But enclave signing is **slow** and not suitable for signing at scale
- HTTP Message Signatures need a signature on every request

jkt-jwt — Why It's Useful

The solution

- Enclave key delegates to a fast **ephemeral** software key via JWT
- Trust On First Use (TOFU) model — like `hwk`, but with delegation
- **Persistent identity**: stable `urn:jkt:<hash-algorithm>:<thumbprint>` identifier (JWK Thumbprint URI) tied to the enclave key
- **Fast signing**: HTTP requests signed with the ephemeral key in software
- **Best of both worlds**: hardware-rooted identity + software signing speed

jkt-jwt — How It Works

1. Enclave generates long-lived identity key pair (hardware-protected)
2. Device generates ephemeral signing key pair (in software)
3. Enclave signs JWT binding ephemeral key via `cnf` claim
4. HTTP requests signed with the fast ephemeral key
5. JWT proves the ephemeral key was authorized by the enclave key

```
Signature-Key: sig=jkt-jwt;jwt="eyJ..."
```

jkt-jwt — JWT Structure

Header — contains the enclave/identity key:

- `typ` : `jkt-s256+jwt` (encodes hash algorithm)
- `alg` : signature algorithm of enclave key
- `jwk` : enclave public key (signs this JWT)

Payload — delegates to ephemeral key:

- `iss` : `urn:jkt:sha-256:<thumbprint>` (identity)
- `iat` , `exp` : validity window
- `cnf.jwk` : ephemeral key for HTTP signing

Open Questions

- **Why a separate header vs. Signature-Input parameter?**
  - Extensible dictionary structure; schemes have varying parameter sets; avoids overloading Signature-Input

Questions / Discussion

- dick.hardt@gmail.com
- draft-hardt-httpbis-signature-key
- https://github.com/dickhardt/signature-key